

Spark Scheduling on Heterogeneous Wimpy Nodes

Estella Xu

Advisor: Prof. Andrew Crotty

Master's Degree Project

June 2026

Abstract

Distributed data analytics platforms like Spark are typically deployed on homogeneous, server-grade clusters. Recent work argues that low-power “wimpy” nodes can still provide fast performance on OLAP queries while requiring significantly less cost and energy, suggesting that incorporating them into a heterogeneous Spark cluster can improve overall cost and efficiency. However, existing heterogeneous schedulers assume server-grade hardware. In this project, we study Spark scheduling on a heterogeneous cluster of wimpy nodes. We introduce an operation-aware scheduling strategy that decides whether to include the slow node in a stage. Across eight TPC-H queries, this strategy improves median runtime by up to 13% over the next-best configuration and 67% over the default.

Artifacts. Project orchestration and testing code is available [here](#). The modified Spark is available [here](#).

Contents

1	Introduction	1
2	Preliminaries	1
2.1	Raspberry Pis and Operating Systems	1
2.2	Spark	2
2.3	Java Virtual Machines (JVMs)	3
3	Cluster Setup	4
3.1	Hardware	4
3.2	Data	5
3.3	Orchestration	5
4	Baseline Configuration	5
4.1	OS-JVM Combinations	5
4.2	JVM Options	6
4.3	Spark Configuration	7
5	Modified Spark Scheduling	9
5.1	Weighted Placement	9
5.2	Operation-Aware Placement	9
6	Experimental Results	11
6.1	Methodology	11
6.2	Results	11
6.3	Discussion	13
7	Future Work	14
8	Conclusion	15

A	Isolated Queries	17
B	Select TPC-H Queries and Logs	17

1 Introduction

Apache Spark ([1]) is a widely used distributed data processing framework. Spark is typically deployed on homogeneous, server-grade hardware. In contrast, “wimpy” nodes, which are low-power, low-cost single-board computers, offer an alternative platform for working with analytical workloads. Recent work ([4]) indicates that wimpy nodes have comparable runtime with significant decrease in cost and energy consumption in OLAP queries, suggesting that incorporating them into a heterogeneous Spark cluster could improve overall efficiency. However, existing methods for heterogeneous scheduling in Spark ([10]) only consider the case where hardware is server-grade.

In this project paper, we show several experiments optimizing Spark scheduling for heterogeneous wimpy node clusters, providing a proof of concept for future improvements.

In Section 2, we discuss preliminary concepts in Spark scheduling. In Section 3, we discuss experimental setup and orchestration. In Section 4, we consider baseline optimizations for Spark on wimpy nodes. In Section 5, we describe different modifications to scheduling. In Section 6, we go through experiment results. Finally, in Section 7, we cover additional areas to explore.

Project orchestration and testing code is available [here](#). The modified Spark is available [here](#).

2 Preliminaries

2.1 Raspberry Pis and Operating Systems

The Raspberry Pi ([7]) is a low-cost, low-power single-board computer. This project uses two generations: the Pi 3B+ and the Pi 5 (details are outlined in Section 3). The Pi 5 is significantly faster and has far more memory than the Pi 3B+, but both are still considered wimpy in comparison.

Because of their limited hardware capabilities, Raspberry Pi SBCs offer different lighter-weight operating systems. The official Raspberry Pi OS is shipped with background services included. An alternative is DietPi, a lighter-weight OS advertising 67% the idle RAM usage and 40% the disk usage of Raspberry Pi OS ([5]). We compare these for Spark in Section 4.

2.2 Spark

Spark ([1]) is a distributed data processing framework that schedules and executes computation over partitioned datasets across a cluster. We describe the relevant components below.

2.2.1 Overview

Spark uses a driver-executor architecture. The **driver** runs the user's application; it hosts the `SparkContext`, builds the logical and physical query plans, and contains the scheduler. The **executors** carry out the processing in JVM processes.

The **cluster manager** supplies executors and allocates resources within the cluster. We use **standalone mode**, in which the master process tracks available machines and launches executors for the driver.

The data structure that Spark uses to process its data is the **Resilient Distributed Dataset** (RDD). RDDs are immutable, partitioned collections of records that can be distributed across executors and processed in parallel. They track their lineage, i.e., the sequence of transformations that produced them, to provide fault tolerance: a lost partition can be recomputed from its lineage. Each **partition** of an RDD is the basis for a unit of work – each task processes one partition – so the number of partitions determines how much work can happen in parallel. Spark SQL compiles `DataFrame` and SQL operations to RDD transformations, which the scheduler then executes.

2.2.2 Spark Scheduling

Suppose we want to run a SQL query. We submit this as a **job** (triggered by an action such as `.show()`). Spark's query optimizer produces a physical execution plan, and the `DAGScheduler` subsequently divides the plan into **stages**. Stage boundaries occur whenever a shuffle is required, i.e., data must be redistributed before computation can continue.

Stages can be one of two kinds. A `ShuffleMapStage` reads an input, either from a file or from a previous shuffle, and writes its output to a shuffle. A `ResultStage` takes in a shuffle output produced by a previous stage and finishes computation to output the final result. For example, a

file scan followed by an aggregation will first have a map stage that scans the table and writes the partial aggregates to a shuffle, and a reduce stage that reads those and combines them into a final result. We will further categorize stage types in Section 5.2.

Each stage has a **TaskSet** containing all **tasks** for that stage. Tasks each correspond to a partition of data, and all tasks in a stage are independent and can be executed in parallel. Tasks are the unit of work that the scheduler assigns to executors.

The TaskSet is submitted to `TaskSchedulerImpl`, which is responsible for mapping tasks to available executors. Each time an executor has resources, the cluster calls `resourceOffers()`, which receives a list of `WorkerOffer` objects from the executors, each containing information such as ID, host, and number of cores. We refer to these as “offers”. For each TaskSet, the scheduler calls `resourceOfferSingleTaskSet()`, which assigns tasks to offers.

Spark assigns tasks based on **locality**. Each task prefers being assigned to the host that has the largest share of the data it must read. The task is assigned to the executor with the best available locality level: `PROCESS_LOCAL` (the same executor), `NODE_LOCAL` the same host, all the way to `ANY`. Spark by default allows delayed scheduling, where the scheduler can wait up to 3s for a preferred executor to become available before falling back to a less local placement.

In standard Spark, the offers within each locality tier are shuffled randomly to avoid starvation. We will discuss alternate orderings for heterogeneous environments in Section 5.1.

2.3 Java Virtual Machines (JVMs)

Spark executors run as JVM processes. The JVM heap is where all of the JVM’s objects, including Spark’s in-memory data structures, are allocated. Spark’s memory manager partitions the JVM heap into three regions. First, the **reserved** memory (300 MB) stores Spark’s internal objects. The remaining heap is split into the **user memory** and the **on-heap unified memory pool**. User memory stores user-defined data structures, internal metadata, and RDD dependency information. The on-heap unified memory is further split into **storage memory** (holding all cached data and broadcast variables) and **execution memory** (holding all objects required during execution such

as intermediate hash tables and shuffle buffers). The two regions in the on-heap unified memory can borrow from one another if blocks are not fully used.

Multiple JVM implementations exist, and we will test three of them. OpenJDK is the open-source implementation built on the HotSpot JVM architecture. Eclipse OpenJ9 ([8]) is built on OpenJ9 rather than HotSpot and advertises higher start-up performance and lower memory consumption. Liberica ([3]) is a distribution built on OpenJDK from BellSoft and advertises faster performance than OpenJ9 ([2]). We test each of these JVMs in Section 4.

3 Cluster Setup

3.1 Hardware

The cluster consists of three machines. The driver is my personal computer, responsible for submitting jobs and hosting the Spark master process. The two worker nodes are:

- **Raspberry Pi 5** with 16 GB RAM, 2.4 GHz quad-core Arm Cortex-A76 processor, and Gigabit Ethernet with a 64 GB SD card. We configure this to be four 1-core executors with 3 GB of heap each. Swap space is disabled.
- **Raspberry Pi 3b+** with 1 GB RAM, 1.4 GHz quad-core Arm Cortex-A53 processor, and 300 Mbps Ethernet with a 64 GB USB stick. We configure this to be one 4-core executor with 512 MB of heap. Swap space is disabled.

All nodes are connected via a local gigabit switch. We split the Pi 5 into four executors to imitate having more nodes. The Pi 3B+ cannot be split further because the minimum execution memory is 450 MB, and we still need sufficient space for non-heap memory, overhead, and OS processes (see 4.2 for more on executor memory).

3.2 Data

TPC-H data ([9]) was generated at scale factors 1, 2, and 3 using **dbgen** ([6]) and stored as `.tbl` files. Data is replicated locally on both worker nodes so each executor can read its assigned partition directly from local disk. We initially set up NFS but found it to be a confounding variable due to network variance. Local data replication ensures that scan stages are consistent. Spark reads the files via its CSV reader with schemas defined to match the TPC-H specifications.

3.3 Orchestration

Cluster management and experiment execution are automated with **Ansible** for easy configuration. The playbook `run-tpch-scala.yml` runs the entire benchmark and data collection: it starts `pidstat` monitoring on the worker nodes, submits the Spark job in standalone mode with the desired configuration flags, and collects logs. Experimental configurations can be easily modified with the parameter dictionaries. A separate playbook, `find-r.yml`, runs a calibration for the per-operator slowdown ratios used by operator aware scheduling (Section 5.2).

4 Baseline Configuration

Because we are in a resource-constrained environment, we begin by modifying some baseline configurations to optimize hardware usage before adding any scheduling. We tested configurations on the Pi 3B+.

4.1 OS-JVM Combinations

Figure 1 shows the runtimes for Q1, 3, 5, and 6 for various combinations of OS and JVM. The DietPi-Liberica combination achieves consistently low runtime and is used for subsequent experiments.

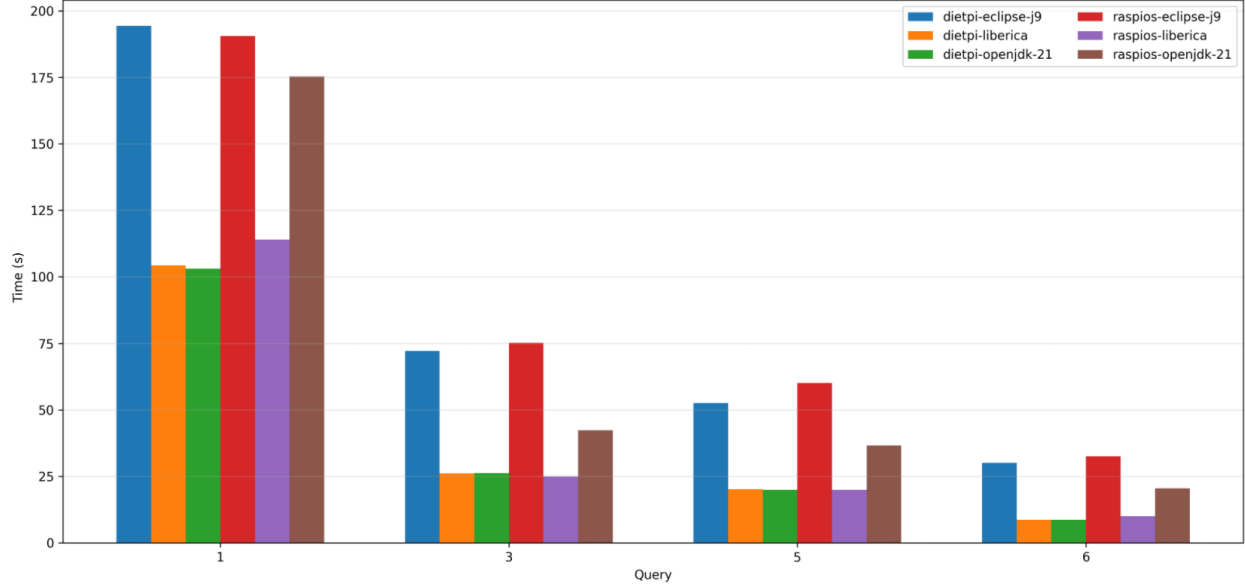


Figure 1: OS-JVM Configuration Comparison

4.2 JVM Options

When submitting a Spark job, the user can set additional options for the JVM. There are two of note in this project:

- Garbage collector.** Figure 2 compares G1GC, ParallelGC, and SerialGC on Q5. SerialGC wins in both runtime and peak memory used. Similar results are reflected in the other queries. This is likely due to the fact that the heap is small enough that a single collection thread can sufficiently manage it, so we do not need the additional overhead and parallelism of G1GC and ParallelGC.
- Initial heap size.** Documentation and tuning guides typically suggest setting the initial heap size to equal the heap size (set via `spark.executor.memory`) with the argument that having the entire heap set aside reduces the overhead needed for the heap to grow. However, in our already memory-constrained scenario, committing the entire heap up front reserves memory that the executor may never use, putting more pressure on off-heap memory and OS processes. Figure 3 shows the measured runtime and peak RSS memory used for Q5 across

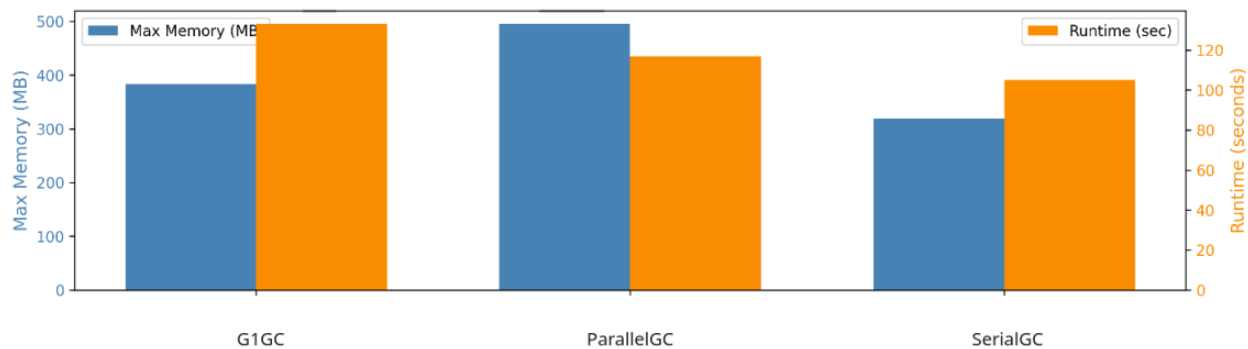


Figure 2: Q5 Garbage Collection (Max Memory and Runtime)

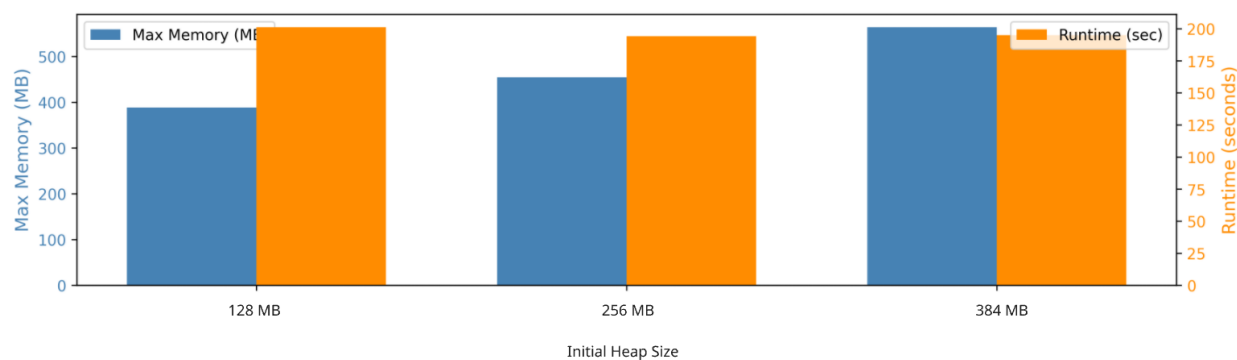


Figure 3: Q5 Initial Heap Size

three different values for initial heap size. We see no runtime benefit, and as the initial heap size increases, as does the peak. Similar results hold for the other queries. So, we set this parameter to the default (which does not set aside the extra memory) and allow the heap to grow as needed.

4.3 Spark Configuration

Now, we describe some configuration parameters from Spark. We test these with the entire cluster as well as with just the Pi 5.

- `sql.shuffle.partitions`. Shuffle partitions are the partitions that intermediate results get written to. Figure 4 shows that more than 16 partitions leads to relatively similar runtime. We

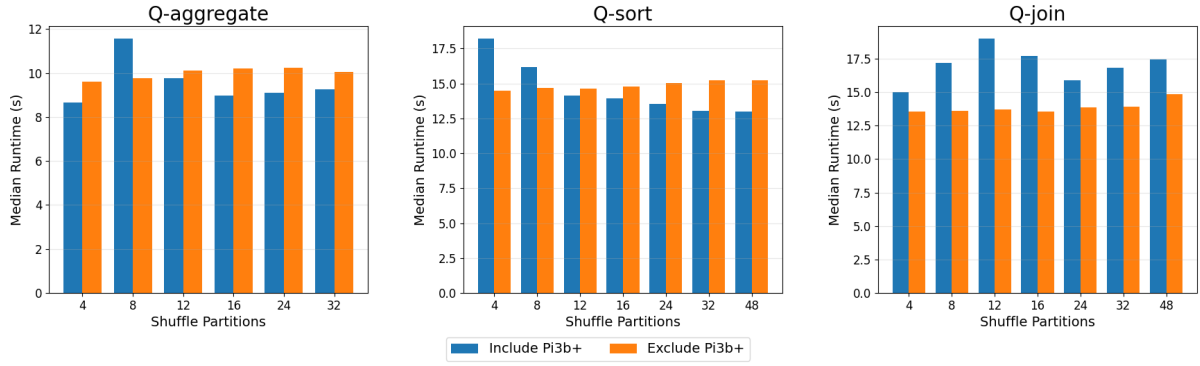


Figure 4: Shuffle Partitions vs. Runtime

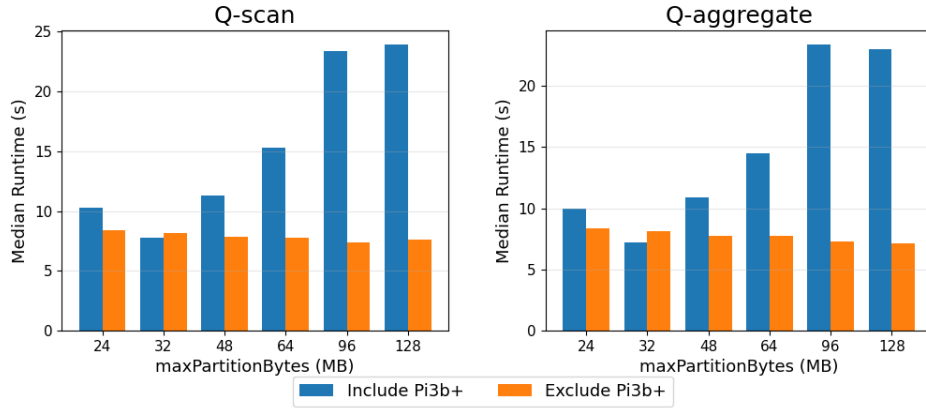


Figure 5: maxPartitionBytes vs. Runtime

set this parameter to 48; `sql.adaptive.enabled` is set to true, allowing Spark to decrease the number of partitions if needed.

- `sql.files.maxPartitionBytes`. This determines the maximum number of bytes to pack in a single partition when reading files. As mentioned in Section 2.2.2, each task corresponds to a partition of data. For example, a 760 MB file with this parameter set to 32 MB would take around 23 tasks to fully scan. Larger partitions means fewer, heavier tasks. Figure 5 shows that increasing this parameter hurts when including the Pi 3B+, since larger partitions cannot be split and will fill up the entire heap and will require spilling. We set this to 32 MB.

- `executor.memory`: We assign 512 MB to the executor Pi 3B+ and 3 GB to each of the four executors on the Pi 5. These values ensure that the heap combined with the memory overhead and OS processes do not exceed the physical RAM of the node and trigger OOM failures. Spark does not natively allow different workers to have different executor memory; changes are made in `StandaloneSchedulerBackend.scala`, `Master.scala`, and `ApplicationDescription.scala` to support passing custom per-worker executor memory to `spark-submit`.

Moving forward, we use these configurations across all experiments.

5 Modified Spark Scheduling

5.1 Weighted Placement

Our initial modification to offer shuffling is simply prioritizing stronger nodes, i.e., the workers corresponding to the Pi 5. Offers presented to the task scheduler are sorted based on data locality and a hardcoded weight, e.g. a weight of 4 for the Pi 5 workers and a weight of 1 for the Pi 3B+ workers. Within each data locality and weight tier, offers are shuffled randomly to avoid starvation.

However, weighting only occasionally yields actual change in the scheduling. Most time-consuming stages consist of many tasks, and since there are only 5 workers in our system, all workers are used anyways. Moreover, tasks in the same stage are effectively equivalent up to data partition, meaning that each time the task scheduler called, it is only working with tasks of the same type. The only way to dynamically assign the tasks themselves would be to look at partitions and place partitions that would be “easier” on the Pi 3B+, but that requires reshuffling the data and determining which partitions are “easier”.

5.2 Operation-Aware Placement

Next, we look at whether or not to include or exclude the Pi 3B+ for a given stage.

Table 1: Stage Types for Operation Aware Placement

Stage Type	readsShuffle	writesShuffle	readsRange	r (SF 1)	r (SF 2)	r (SF 3)
ScanHeavy	No			4.78	6.52	6.99
AggReduce	Yes	No	No	5.10	5.37	4.69
SortReduce	Yes	No	Yes	5.20	7.36	7.82
JoinReduce	Yes	Yes		7.36	5.75	6.6

To formalize this, we first categorize tasks by how much they benefit or hurt as a result of including the 3b+. We define four stage types, outlined in Table 1. A ScanHeavy stage does not read shuffle input, but rather data from a file. The other three stages read shuffle input. A JoinReduce stage both reads and writes a shuffle, indicating intermediate data. A SortReduce reads a range-partitioned shuffle. An AggReduce reads a hash-partitioned shuffle without range partitioning. We determine which type a stage falls under by reading logged shuffle dependencies in Spark’s DAGScheduler to extract whether the operation reads shuffled data as an input (readsShuffle: indicates a reduce type), uses range partitioning (readsRange: indicates ordering), or writes shuffle output (writesShuffle: indicates writing to an intermediate result).

For each stage, define the number of waves to be

$$w = \left\lceil \frac{\text{number of tasks}}{\text{number of Pi 5 workers}} \right\rceil.$$

For each stage type, let

$$r = \frac{t_3}{t_5}$$

where t_3 is the execution time per task on a Pi 3B+ worker and t_5 is the execution time per task on a Pi 5 worker. At runtime, the task scheduler includes the Pi 3B+ if $w > r$, i.e., the stage is large enough that the slowness of the Pi 3B+ does not bottleneck the overall computation. The last three columns of Table 1 shows the values of r for SF 1, 2, and 3 used for subsequent benchmark results. These r values are from a diagnostic test that runs the queries in Appendix A and parses the Spark event logs prior to running the benchmark, aiming to isolate each operation to calculate slowdown from the Pi 3B+. This is available in `find-r.yml`.

Table 2: Runtime Change from Operation-Aware vs. Baselines

Query	Operation Aware	vs. Next-Best (%)	vs. Default (%)
1	59.168	-2.97	-2.97
3	22.192	13.18	15.77
4	17.874	1.52	1.52
5	12.419	9.31	24.32
6	21.492	0.77	2.72
13	5.538	-2.89	66.61
14	17.893	9.82	15.51
19	16.485	9.68	12.72

6 Experimental Results

6.1 Methodology

For each scale factor, we first run the `find-r.yml` playbook to obtain the slowdown ratios immediately before the benchmark so that it reflects the same conditions as the measured run. Then, we execute each query repeatedly under each scheduling configuration. Between each query run, we clear the Spark cache, unpersist residual RDDs, invoke GC, and pause so that runs are independent from their predecessors. The first run of each configuration is discarded as a warm-up, and we report the median over the remaining. We use SF 2 on all queries except Q5, where we scale down to SF 1 to avoid OOM failures, and Q1 and Q6, where we scale up to SF 3.

6.2 Results

Figure 6 shows the runtime across eight TPC-H queries with default (unweighted) scheduling, weighted scheduling, Pi 5 only, and operation-aware placement. Table 2 shows the median runtime achieved with operation-aware placement along with the percentage decrease in runtime relative to the next-best configuration and the default configuration. Note that the increases in runtime are all within the variance of the runs. Across all queries, operation-aware yields a median runtime improvement of 5.4% over the next-best configuration and 14.1% over the default configuration, with maximum improvements of 13.2% and 66.6%, respectively.

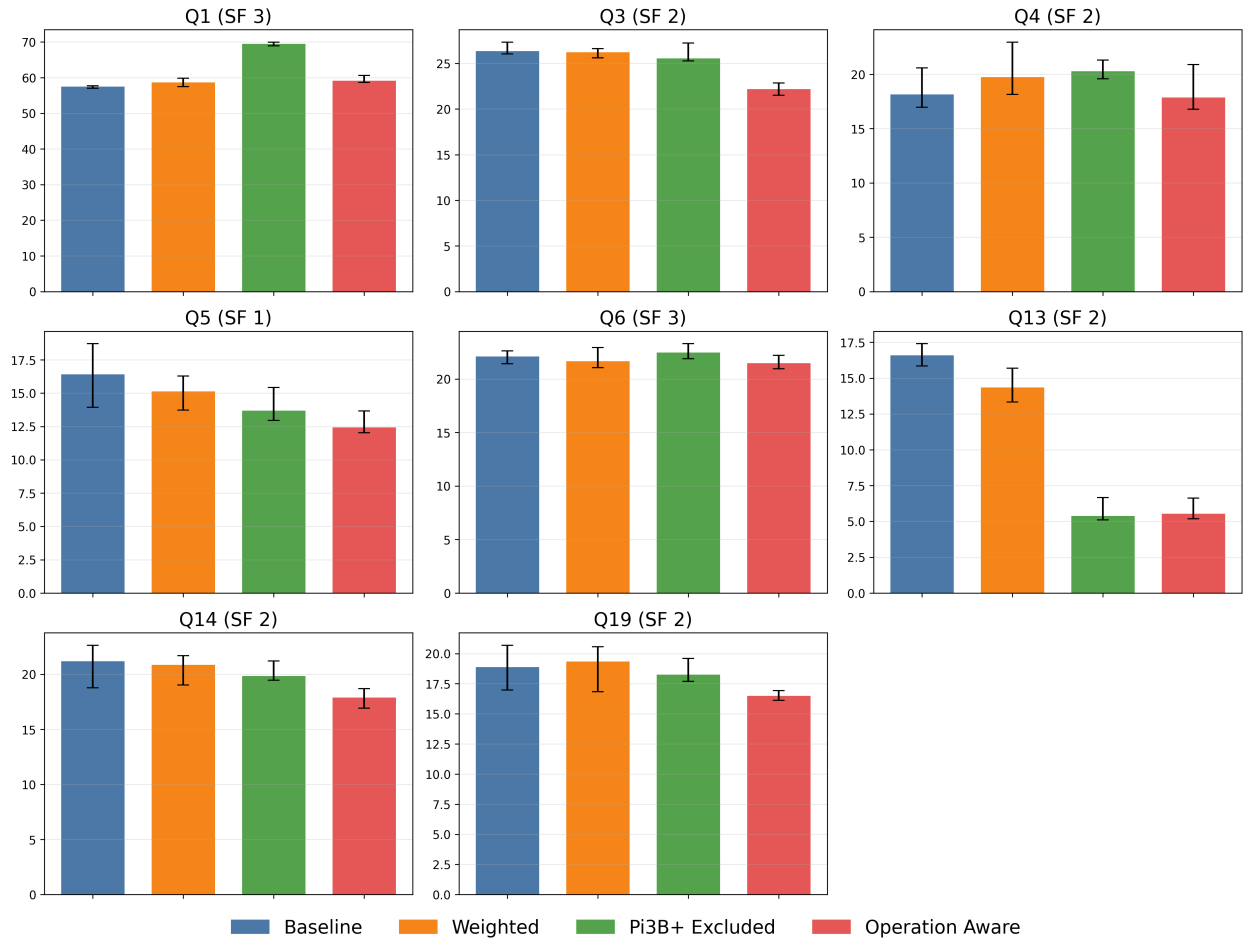


Figure 6: TPC-H Queries by Scheduling Strategy vs. Runtime (s)

6.3 Discussion

Now, we discuss a few of the results seen in Figure 6 that demonstrate the impacts of each placement strategy. Appendix B shows the stage and worker task breakdown for Q1 and Q13.

Q1: excluding the Pi 3B+ is slower than including. Q1’s `lineitem` scan consists of 69 tasks (`lineitem` is roughly 2.3 GB in SF3, and with each partition being 32 MB, we have approximately 69 partitions). So, we have $\lceil \frac{69}{4} \rceil = 18$ waves. The baseline (i.e., equal weights with Pi 3B+ and Pi 5 workers) places 12 of those tasks to the Pi 3B+’s 4 cores, each taking approximately 18.7s. The remaining 57 are allocated to the 4 Pi 5 workers, each taking 3.9s. In parallel, the entire stage takes $\max\{\lceil 57/4 \rceil \cdot 3.9, \lceil 12/4 \rceil \cdot 18.7\} = 58.5$ s. On the other hand, if the 3b+ worker is excluded, all 69 tasks are given to the Pi 5 workers, taking $\lceil 69/4 \rceil \cdot 3.9 = 70.2$ s. In this case, the workers on the two Pis can finish at comparable time since the number of waves is much greater than r_{scan} , so there is plenty of work for the Pi 5 to do while overlapping with the slow Pi 3B+’s contribution.

Q13: excluding the Pi 3B+ is significantly faster than including. Compared to Q1, Q13 shows the opposite effect. It consists of only smaller stages: a scan with 11 tasks (customers and orders are roughly 375 MB total for SF 2), a join with 9 tasks, and a final aggregation with 1 task. The baseline places 7 tasks on the Pi 5 at 1.1s each and 4 on the Pi 3B+ at 7s each, so that the stage takes 7s total. On the other hand, excluding the Pi 3B+ places all 11 tasks on the Pi 5, finishing $\lceil 11/4 \rceil = 3$ waves at 1.1s each for a total of 3.3s. This is less than the include configuration, which spends too much time waiting on the Pi 3B+’s single wave of slow tasks, since there are too few tasks to overlap tasks more. This is further compounded by the join stage, since the shuffle partitions end up being placed more on the Pi 3B+. This is something to explore in the future (see Section 7).

Q13: weighting is somewhat faster than the default, but not as fast as exclusion. The weighted ordering presents the Pi 5 workers ahead of the Pi 3B+, so the Pi 3B+ receives a total of 3 tasks instead of 4, resulting in a makespan of 5.25s.

Overall, including the Pi 3B+ helps when a stage has more Pi 5 waves than its slowdown ratio, so that there is enough work to do on the Pi 5 workers to overlap with the Pi 3B+'s slow runtime. These results show that the operation aware placement strategy is as good as, if not better than, the next best strategy.

7 Future Work

There are several possible next steps:

1. *Scaling out.* By adding more nodes, we can get a more realistic idea of how a heterogeneous cluster of wimpy nodes would function. Simple weighted scheduling may now yield more differences. If there are a sufficient number of workers, not all workers will need to be allocated for each stage, meaning that prioritization will impact placement.
2. *Improved file storage.* Duplicating the data on each node is not realistic and also constrains us to the available disk on the smallest node. We can potentially explore Hadoop Distributed File System or storing data with an object store.
3. *Scaling up.* If we are able to scale out and improve file storage, we can then test with larger datasets.
4. *Heterogeneity in different dimensions.* With the simple cluster, the Pi 5 beats the Pi 3B+ in every category, meaning that any heavier operation will prioritize the Pi 5. However, if we have nodes that differ in more complex ways, for example, more memory and less compute, the placement decision becomes more interesting than just include/exclude. For example, we could try more interesting strategies like filter pushdown, where we offload a memory-heavy scan to a high-memory node.
5. *Modify the physical plan, locality, and shuffle placement.* This project only chooses whether or not a slow node participates; it does not control where the shuffle output is written to.

As we saw for Q13, including the slow node in the first map stage can also slow down the downstream reduce stage, since locality puts reduce tasks on whichever host holds the largest share of the shuffle input. We could potentially steer shuffle output placement towards the faster node and also explore the tradeoff of using or excluding data locality as part of the scheduling decision.

6. *Adaptive cost-finding for r .* As described in Section 5.2, we calculate the slowdown ratio with a script prior to each benchmark. A more efficient and adaptable approach would be to estimate r , either online, by observing per-task runtimes as each stage executes and updating for future stages, or analytically, by modeling task cost.

8 Conclusion

We have shown that Spark’s scheduling decisions that are built for homogeneous clusters do not perform well on a heterogeneous wimpy cluster. A single slow node can stretch out the makespan of stages with fewer tasks, but can speed up the runtime of larger stages. In this project, we first established baseline configurations to maximize options in our already constrained environment. We then introduced an operation-aware placement strategy that decides whether to include or exclude the slow executor for each stage, by comparing the number of tasks to the calculated per-operator slowdown ratio. Across eight TPC-H queries, this policy performs as well as, if not better than, the baselines, improving median runtime by up to 13% over the next-best configuration. This provides a proof of concept, and Section 7 outlines how we can extend and further optimize to larger, more realistic clusters.

References

- [1] Apache Spark. *Apache Spark Documentation*. 2026. URL: <https://spark.apache.org/docs/latest/>.
- [2] BellSoft. *HotSpot vs. OpenJ9 — Choosing the Right JVM*. 2024. URL: <https://bell-sw.com/announcements/2022/06/28/hotspot-vs-openj9-performance-comparison/>.
- [3] Bellsoft. *Liberica JDK Documentation*. 2026. URL: <https://docs.bell-sw.com/liberica-jdk/>.
- [4] Andrew Crotty et al. “The Case for In-Memory OLAP on ‘Wimpy’ Nodes”. In: *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE. 2021, pp. 732–743.
- [5] DietPi. *DietPi Benchmark Statistics*. 2026. URL: <https://dietpi.com/stats.html>.
- [6] electrum. *TPC-H dbgen*. GitHub repository. 2026. URL: <https://github.com/electrum/tpch-dbgen>.
- [7] Raspberry Pi. *Raspberry Pi*. 2026. URL: <https://www.raspberrypi.com/>.
- [8] The Eclipse Foundation. *Eclipse OpenJ9*. 2026. URL: <https://eclipse.dev/openj9/>.
- [9] Transaction Processing Performance Council (TPC). *TPC Benchmark H (Decision Support) Standard Specification, Revision 2.17.3*. Tech. rep. <https://www.tpc.org/tpch/>. 1999.
- [10] Luna Xu et al. “A heterogeneity-aware task scheduler for spark”. In: *2018 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE. 2018, pp. 245–256.

A Isolated Queries

```
1 SELECT l_orderkey
2 FROM lineitem
3 WHERE l_shipdate <= DATE '1998-09-02'
```

Listing 1: Q_{Scan}

```
1 SELECT l_partkey, SUM(l_quantity)
2 FROM lineitem
3 GROUP BY l_partkey
```

Listing 2: Q_{Agg}

```
1 SELECT l_orderkey, l_shipdate
2 FROM lineitem
3 ORDER BY l_shipdate
```

Listing 3: Q_{Sort}

```
1 SELECT COUNT(*)
2 FROM lineitem l
3 JOIN orders o ON l.l_orderkey = o.o_orderkey
```

Listing 4: Q_{Join}

B Select TPC-H Queries and Logs

```
1 SELECT
2     l_returnflag,
3     l_linestatus,
4     SUM(l_quantity) as sum_qty,
5     SUM(l_extendedprice) as sum_base_price,
6     SUM(l_extendedprice * (1 - l_discount)) as sum_disc_price,
7     SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) as sum_charge,
8     AVG(l_quantity) as avg_qty,
9     AVG(l_extendedprice) as avg_price,
10    AVG(l_discount) as avg_disc,
11    COUNT(*) as count_order
12 FROM lineitem
13 WHERE l_shipdate <= date '1998-12-01' - interval '90' day
14 GROUP BY l_returnflag, l_linestatus
15 ORDER BY l_returnflag, l_linestatus
```

Listing 5: Q1

Table 3: Task placement and execution time across scheduling configurations for Q1

Stage	Type	Host	# of Tasks	Time per Task (ms)
Equal Weight				
1	ScanHeavy	Pi 5	57	3915.4
1	ScanHeavy	Pi 3B+	12	18712.9
2	AggReduce	Pi 5	1	53.0
Exclude Pi 3B+				
1	ScanHeavy	Pi 5	69	3912.8
2	AggReduce	Pi 5	1	41.0

```

1 SELECT
2     c_count,
3     COUNT(*) AS custdist
4 FROM (
5     SELECT
6         c_custkey,
7         COUNT(o_orderkey) AS c_count
8     FROM customer
9     LEFT OUTER JOIN orders
10        ON c_custkey = o_custkey
11        AND o_comment NOT LIKE '%special%requests%'
12    GROUP BY c_custkey
13 ) c_orders
14 GROUP BY c_count
15 ORDER BY custdist DESC, c_count DESC

```

Listing 6: Q13

Table 4: Task placement and execution time across scheduling configurations for Q13

Stage	Type	Host	# of Tasks	Time per Task (ms)
Equal Weight				
67	ScanHeavy	Pi 5	7	1048.4
67	ScanHeavy	Pi 3B+	4	6996.0
70	JoinReduce	Pi 5	1	251.0
70	JoinReduce	Pi 3B+	8	2735.6
74	AggReduce	Pi 5	1	19.0
Weighted Priority				
67	ScanHeavy	Pi 5	8	1057.6
67	ScanHeavy	Pi 3B+	3	6736.3
70	JoinReduce	Pi 5	2	311.5
70	JoinReduce	Pi 3B+	7	2910.9
74	AggReduce	Pi 5	1	19.0
Exclude Pi 3B+				
67	ScanHeavy	Pi 5	11	1084.1
70	JoinReduce	Pi 5	9	425.6
74	AggReduce	Pi 5	1	12.0